ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

старший преподаватель		Н.В. Путилова
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №3

Объектно-реляционные базы данных. Проектирование и создание

по курсу: ПРОЕКТИРОВАНИЕ БАЗ ДАННЫХ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. №	4236		А.В. Панченко
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2025

Цель работы: научиться проектировать физическую модель базы данных в третьей нормальной форме с использованием наследования и пользовательских типов, а также писать скрипты для создания базы данных с учетом корректной работы внешних ключей через триггеры.

Описание задания:

Спроектировать физическую модель базы данных, находящуюся в третьей нормальной форме и включающей наследование и хотя бы один пользовательский типов соответствии с заданным вариантом. Написать соответствующий скрипт создания базы данных.

Вариант 18:

18. служба доставки с учетом разницы физических и юридических лиц: адрес доставки, контактное лицо, стоимость посылки, диапазон желаемого времени доставки. время доставки фактическое, вес посылки, отметка о доставке, фирма отправитель (у юр. лица есть контактное лицо (человек))

а. все посылки, отправляемые на улицу, в названии которой есть «малая», но это не последнее слово в нем

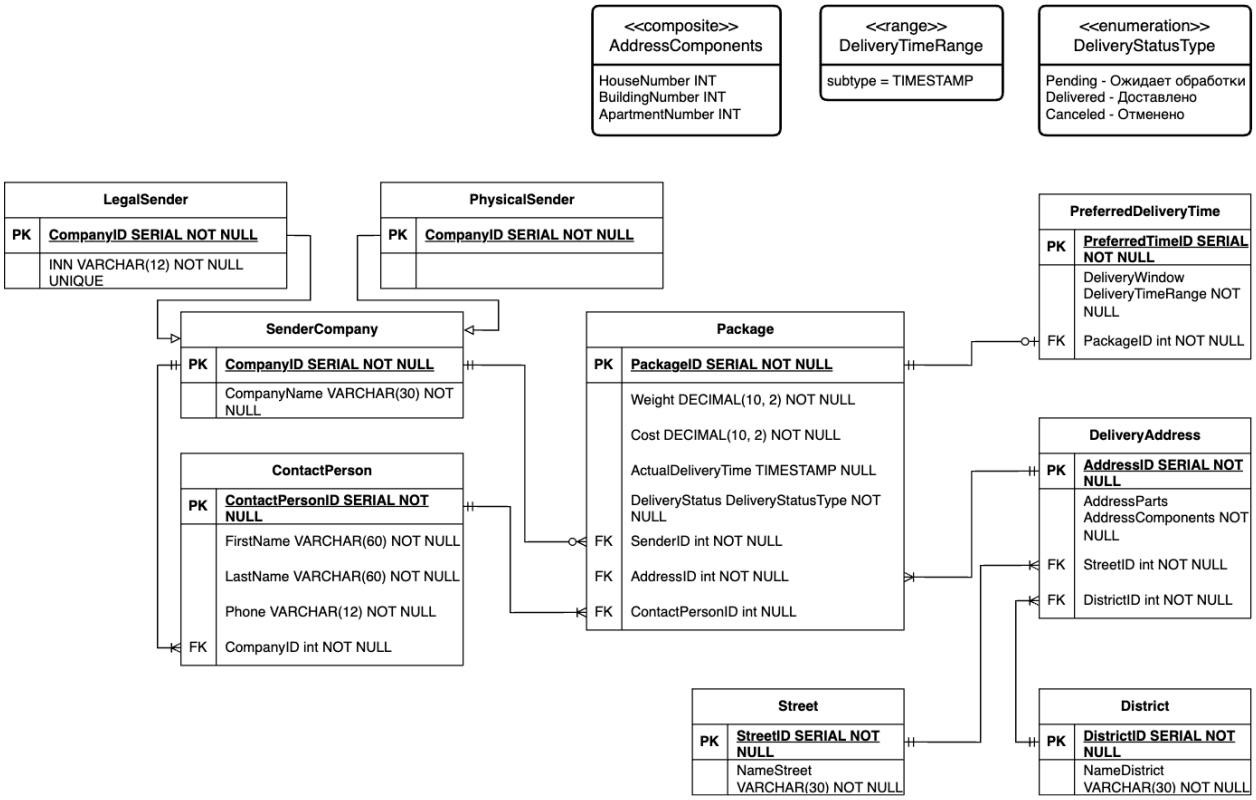
б. контакт(человек) юридического лица, получавшее посылки по разным адресам

в. фирма, отправившая больше среднего посылок для юр. лиц.

г. посылка с самым маленьким весом

д. контактное лицо, не получавшее посылок в 2022 году

Физическая модель БД:



Скрипт создания БД:

```
-- Создание пользовательских типов
CREATE TYPE DeliveryStatusType AS ENUM ('Pending', 'Delivered', 'Canceled');
CREATE TYPE DeliveryTimeRange AS RANGE (subtype = TIMESTAMP);
CREATE TYPE AddressComponents AS (
    HouseNumber INT,
    BuildingNumber INT,
    ApartmentNumber INT
);

-- Основные таблицы
CREATE TABLE SenderCompany (
    CompanyID SERIAL NOT NULL PRIMARY KEY,
    CompanyName VARCHAR(30) NOT NULL
);

-- Таблицы-наследники
CREATE TABLE LegalSender (
    INN VARCHAR(12) NOT NULL UNIQUE,
    PRIMARY KEY (CompanyID)
) INHERITS (SenderCompany);

CREATE TABLE PhysicalSender (
    PRIMARY KEY (CompanyID)
) INHERITS (SenderCompany);

-- Таблица контактных лиц
CREATE TABLE ContactPerson (
    ContactPersonID SERIAL NOT NULL PRIMARY KEY,
```

```
    FirstName VARCHAR(60) NOT NULL,
    LastName VARCHAR(60) NOT NULL,
    Phone VARCHAR(12) NOT NULL,
    CompanyID INT NOT NULL
);

-- Остальные таблицы
CREATE TABLE District (
    DistrictID SERIAL NOT NULL PRIMARY KEY,
    NameDistrict VARCHAR(30) NOT NULL
);

CREATE TABLE Street (
    StreetID SERIAL NOT NULL PRIMARY KEY,
    NameStreet VARCHAR(30) NOT NULL
);

CREATE TABLE DeliveryAddress (
    AddressID SERIAL NOT NULL PRIMARY KEY,
    AddressParts AddressComponents NOT NULL,
    StreetID INT NOT NULL REFERENCES Street(StreetID) ON UPDATE
CASCADE ON DELETE RESTRICT,
    DistrictID INT NOT NULL REFERENCES District(DistrictID) ON
UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE Package (
    PackageID SERIAL NOT NULL PRIMARY KEY,
    Weight DECIMAL(10,2) NOT NULL,
    Cost DECIMAL(10,2) NOT NULL,
```

```
    ActualDeliveryTime TIMESTAMP NULL,
    DeliveryStatus DeliveryStatusType NOT NULL,
    SenderID INT NOT NULL,
    AddressID INT NOT NULL REFERENCES DeliveryAddress(AddressID) ON UPDATE CASCADE ON DELETE RESTRICT,
    ContactPersonID INT NULL REFERENCES ContactPerson(ContactPersonID) ON UPDATE CASCADE ON DELETE SET NULL
);

CREATE TABLE PreferredDeliveryTime (
    PreferredTimeID SERIAL NOT NULL PRIMARY KEY,
    DeliveryWindow DeliveryTimeRange NOT NULL,
    PackageID INT NOT NULL REFERENCES Package(PackageID) ON UPDATE CASCADE ON DELETE CASCADE
);

-- 1. Триггеры для SenderCompany (UPDATE, DELETE)
CREATE OR REPLACE FUNCTION handle_sender_company_update()
RETURNS TRIGGER AS $$
BEGIN

    -- Обновляем связанные таблицы
    UPDATE ContactPerson SET CompanyID = NEW.CompanyID
    WHERE CompanyID = OLD.CompanyID;

    UPDATE Package SET SenderID = NEW.CompanyID
    WHERE SenderID = OLD.CompanyID;

    RETURN NEW;
```

```
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_sender_company_update
AFTER UPDATE ON SenderCompany
FOR EACH ROW EXECUTE FUNCTION handle_sender_company_update();

CREATE OR REPLACE FUNCTION handle_sender_company_delete()
RETURNS TRIGGER AS $$
BEGIN
  -- Проверяем ссылки перед удалением
  IF EXISTS (SELECT 1 FROM ContactPerson WHERE CompanyID = OLD.CompanyID) THEN
    RAISE EXCEPTION 'Нельзя удалить компанию с существующими контактами (ID: %)', OLD.CompanyID;
  END IF;

  IF EXISTS (SELECT 1 FROM Package WHERE SenderID = OLD.CompanyID) THEN
    RAISE EXCEPTION 'Нельзя удалить компанию с существующими посылками (ID: %)', OLD.CompanyID;
  END IF;

  RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_sender_company_delete
BEFORE DELETE ON SenderCompany
```

```
FOR EACH ROW EXECUTE FUNCTION handle_sender_company_delete();

-- 2. Триггеры для ContactPerson (INSERT, UPDATE)
CREATE OR REPLACE FUNCTION check_contact_person_operations()
RETURNS TRIGGER AS $$
BEGIN
  IF TG_OP = 'INSERT' OR TG_OP = 'UPDATE' THEN
    -- Проверяем существование компании
    IF NOT EXISTS (SELECT 1 FROM SenderCompany WHERE CompanyID = NEW.CompanyID) THEN
      RAISE EXCEPTION 'CompanyID % не существует в SenderCompany', NEW.CompanyID;
    END IF;

    -- Для физических отправителей - только один контакт
    IF EXISTS (SELECT 1 FROM PhysicalSender WHERE CompanyID = NEW.CompanyID) THEN
      IF (SELECT COUNT(*) FROM ContactPerson WHERE CompanyID = NEW.CompanyID) > 1 THEN
        RAISE EXCEPTION 'Физический отправитель может иметь только один контакт';
      END IF;
    END IF;

    RETURN NEW;
  END IF;

  RETURN OLD;
END;
```

```
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_contact_person_operations
BEFORE INSERT OR UPDATE ON ContactPerson
FOR EACH ROW EXECUTE FUNCTION check_contact_person_operations();

-- 3. Триггеры для Package (INSERT, UPDATE)
CREATE OR REPLACE FUNCTION check_package_operations()
RETURNS TRIGGER AS $$
BEGIN
  -- Проверка существования отправителя
  IF NOT EXISTS (SELECT 1 FROM SenderCompany WHERE CompanyID = NEW.SenderID) THEN
    RAISE EXCEPTION 'SenderID % не существует', NEW.SenderID;
  END IF;

  -- Проверка контактного лица (если указано)
  IF NEW.ContactPersonID IS NOT NULL AND
    NOT EXISTS (SELECT 1 FROM ContactPerson WHERE ContactPersonID = NEW.ContactPersonID) THEN
    RAISE EXCEPTION 'ContactPersonID % не существует', NEW.ContactPersonID;
  END IF;

  -- Проверка что контактное лицо принадлежит отправителю
  IF NEW.ContactPersonID IS NOT NULL AND EXISTS (
    SELECT 1 FROM ContactPerson
    WHERE ContactPersonID = NEW.ContactPersonID AND CompanyID != NEW.SenderID
```

```
        ) THEN
            RAISE EXCEPTION 'Контактное лицо не принадлежит указанному отправителю';
        END IF;

        RETURN NEW;
    END;
    $$ LANGUAGE plpgsql;

    CREATE TRIGGER trg_package_operations
    BEFORE INSERT OR UPDATE ON Package
    FOR EACH ROW EXECUTE FUNCTION check_package_operations();
```

Вывод:

В ходе выполнения задания я научился проектировать физическую модель базы данных, соответствующую третьей нормальной форме, с использованием наследования и пользовательских типов данных. Я освоил создание сложных структур таблиц, включая наследование, а также написание SQL-скриптов для реализации базы данных. Особое внимание было уделено корректной работе внешних ключей через триггеры, что позволило обеспечить целостность данных. Этот опыт помог мне лучше понять принципы проектирования баз данных и их реализацию в PostgreSQL, что будет полезно для выполнения более сложных задач в будущем.